

PROJECT REPORT

Retrieval-Augmented Generation Application

College Information Assistant

Submitted by: Praveen K J

Programme: Business Analytics Certification — ASAP Kerala

Date: May 2026

1. Project Title

College Information Assistant — A Retrieval-Augmented Generation (RAG) Application

2. Use Case Selected

Educational Institution Query System. Students and prospective applicants frequently need accurate information about college policies — admission requirements, tuition fees, examination rules, hostel regulations, library timings, and the academic calendar. This application allows users to ask natural language questions and receive precise answers drawn directly from official college documents, without needing to read through multiple policy files manually.

3. Problem Statement

College students and applicants often struggle to find specific policy information buried across multiple documents. Reading entire policy files to find a single fact (e.g., the library fine amount or hostel curfew time) is time-consuming and inefficient. A general-purpose AI chatbot cannot be used for this purpose because it may hallucinate institution-specific facts that it was never trained on.

This project solves the problem by building a RAG system that:

- Retrieves only the relevant portion of the official policy documents
- Grounds every answer in the actual source text

- Refuses to answer questions not covered in the knowledge base
- Cites the source document for every response

4. Description of Knowledge Base

The knowledge base consists of six plain-text (.txt) policy documents created specifically for this application. Each file covers one domain of college administration:

File	Topic	Key Information Covered
admission_policy.txt	Admissions	Application process, required documents, fees, international student requirements, deferral policy
tuition_fees.txt	Fees & Finance	Tuition rates, payment schedule, late fines, scholarships, refund policy
examination_rules.txt	Examinations	Attendance requirements, hall conduct, permitted materials, academic dishonesty, re-evaluation
hostel_rules.txt	Hostel/Residence	Accommodation fees, curfew, guest policy, room rules, maintenance
library_policy.txt	Library	Operating hours, borrowing limits, fines, conduct rules, digital resources
academic_calendar.txt	Academic Calendar	Semester dates, add/drop deadlines, mid-terms, winter break, important dates

All documents were written with sufficient detail (5 sections per file, approximately 300-400 words each) to enable meaningful chunking and accurate retrieval.

5. Data Preparation Steps

5.1 Document Creation

Each knowledge base file was structured with clear section headings (e.g., APPLICATION PROCESS, PAYMENT SCHEDULE) to make content boundaries natural for the text splitter. All information is factual and consistent across documents.

5.2 File Cleaning

- Removed one empty file (requirements.txt — 0 bytes) which was skipped automatically by the loader
- Ensured all files use UTF-8 encoding and plain ASCII text
- Verified no duplicate content exists across files

5.3 Loading Pipeline

The ingestion pipeline uses LangChain's DirectoryLoader with format-specific loaders:

```
SUPPORTED_LOADERS = {  
    '*.txt': TextLoader,  
    '*.md': TextLoader,  
    '*.pdf': PyPDFLoader,  
}
```

The loader uses `silent_errors=True` to skip unreadable files without crashing the pipeline.

6. Chunking Strategy

Parameters Used

Parameter	Value	Reason
chunk_size	500 characters	Large enough to capture full policy statements within a single chunk
chunk_overlap	50 characters	Prevents important sentences from being cut at chunk boundaries
Splitter	RecursiveCharacterTextSplitter	Tries to split on paragraph breaks first, then sentences, then words

Why This Strategy

A chunk size of 500 characters was chosen to capture complete policy statements within each chunk. The `RecursiveCharacterTextSplitter` was selected over a plain character splitter because it tries to split on paragraph breaks first, then sentences, then words — preserving natural language boundaries within each chunk. A 50-character overlap ensures that no important sentence is cut off at a chunk boundary.

Deduplication

Each chunk is assigned a stable MD5-based ID combining the source filename and content hash. Re-running ingestion checks existing IDs in ChromaDB and only adds new chunks, preventing duplicate embeddings.

Code used:

```
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
```

7. Embedding Model Used

Property	Value
Model Name	all-MiniLM-L6-v2
Provider	HuggingFace (sentence-transformers)

Property	Value
Dimensions	384
Library	langchain-huggingface
Runs locally	Yes — no API key required
Speed	Fast — 103 weight files load in under 1 second

This model was selected because it is lightweight, runs entirely locally without an API call, produces high-quality semantic embeddings for short text passages, and is the most widely used model for RAG applications at this scale.

8. Vector Store Used

Property	Value
Vector Store	ChromaDB
Library	langchain-chroma
Storage	Persisted to disk (chroma_db/ folder)
Search Method	Cosine distance similarity
Top-k Results	3 chunks per query

ChromaDB was chosen for its simplicity, zero-configuration setup, and native LangChain integration. It persists the vector index to disk so embeddings do not need to be recomputed on every run. The distance threshold of 1.0 filters out chunks that are too dissimilar to the query.

9. Retrieval Process

The retrieval pipeline follows these steps on every user query:

1. The user's question is converted to a 384-dimensional vector using the same all-MiniLM-L6-v2 embedding model used during ingestion.
2. ChromaDB performs a cosine distance search across all stored chunk embeddings and returns the top 3 closest matches.
3. Chunks with a distance score above 1.0 (too dissimilar) are filtered out.
4. The remaining chunks are returned as (Document, score) tuples, each carrying source file metadata.
5. The chunks are joined with a separator and passed as context to the language model.

A singleton pattern is used so the embedding model and ChromaDB connection are loaded only once per session, avoiding a 1-2 second reload penalty on every question.

10. Prompt Used

The following prompt template was used to instruct the language model:

```
PROMPT_TEMPLATE = """
Answer the question based ONLY on the following context:
{context}
---
Question: {question}
Assistant Answer: (If the answer isn't in the context,
say 'Information not found in documents.')
"""
```

Design Decisions

- ONLY on the following context — prevents the LLM from using its own training knowledge, ensuring all answers are grounded in the documents
- Explicit fallback instruction — tells the model to clearly say when information is missing, rather than guessing
- Minimal prompt — keeps token usage low for the free-tier API
- No conversation history — each query is independent, keeping the system simple and stateless

11. Screenshots / Sample Outputs

[Screenshots to be inserted here showing the terminal running with sample questions and answers]

The application runs in the terminal as shown below (sample):

```
=====
College Information Assistant
=====
Type your question. Type 'exit' to quit.

You: what is the tuition fee?
[retrieval] Loading embedding model...
[retrieval] Ready.

Assistant: The tuition fee varies based on student status:
- Full-time undergraduate: $5,000 per semester
- Part-time (< 3 units): $800 per unit
- Graduate program: $6,500 per semester

Sources: tuition_fees.txt, admission_policy.txt
```

12. Testing Results

The following test questions were used to evaluate the system across all six knowledge base files:

Test Question	Retrieved Topic	Generated Answer (Summary)	Correct?
What is the tuition fee?	tuition_fees.txt	Full-time undergrad: \$5,000/semester; Part-time: \$800/unit; Graduate: \$6,500/semester	Yes
What documents do I need to apply?	admission_policy.txt	High school diploma, certified transcripts, government-issued ID, personal statement, passport photos	Yes
What happens if I cheat in an exam?	examination_rules.txt	Immediate F grade and possible suspension; second offense leads to permanent expulsion	Yes
What is the hostel curfew time?	hostel_rules.txt	Nightly curfew at 9:00 PM; late returns must sign the late entry register	Yes
How many books can I borrow from the library?	library_policy.txt	Maximum 3 books simultaneously for 14 days; renewable once if not reserved by another student	Yes
When does the fall semester start?	academic_calendar.txt	Fall Semester begins September 1, 2026; orientation runs August 25-31	Yes
What is the late payment fine?	tuition_fees.txt	Flat \$100 fine applied after the 15th of the month	Yes
Can I cook in my dorm room?	hostel_rules.txt	No — cooking with electric heaters or gas stoves is forbidden; common kitchen available 7AM-10PM	Yes
What is the library fine for late returns?	library_policy.txt	\$1 per day per book; lost books charged at 3x market value	Yes
Is gym membership provided?	Not found	Information not found in documents.	Yes — correctly identified as missing
What is the notice period for resignation?	Not found	Information not found in documents.	Yes — correctly identified as missing
What is the minimum attendance for exams?	examination_rules.txt	Minimum 75% attendance required in each course	Yes

Summary

Metric	Result
Total questions tested	12
Correct answers	12
Accuracy	100%
Questions correctly marked 'not found'	2 out of 2
Average response time	~2-3 seconds
Source attribution accuracy	100%

13. Challenges Faced

13.1 API Key Quota Exhaustion

The initial Google Gemini API key was accidentally exposed in a screenshot and subsequently exhausted by unauthorized requests. The solution was to switch to the Groq API (llama-3.1-8b-instant), which offers a more generous free tier with 14,400 requests per day.

13.2 Deprecated ChromaDB Import

The `langchain_community.vectorstores.Chroma` import was deprecated in LangChain 0.2.9. This produced warnings and potential future breakage. Resolved by installing the standalone `langchain-chroma` package and updating all imports.

13.3 Negative Relevance Scores

Using `similarity_search_with_relevance_scores()` with the `all-MiniLM-L6-v2` model produced scores outside the expected 0-1 range (including negative values). This caused valid chunks to be filtered out incorrectly. Resolved by switching to `similarity_search_with_score()` which returns raw cosine distances, and inverting the threshold logic (lower score = more relevant).

13.4 Too Few Chunks

The initial chunk size of 500 characters produced only 6 chunks from 6 files, meaning each file was essentially stored as one chunk. This made retrieval inaccurate for specific sub-questions. Reduced chunk size to 200 characters to produce more granular chunks and improve retrieval precision.

13.5 Incorrect Model Name

The model string `'gemini-1.5-flash'` and `'gemini-1.5-flash-latest'` both returned 404 errors. Used the `ListModels` API to discover the correct available model names for the account's API version.

14. Possible Improvements

Improvement	Description	Impact
Streamlit Web UI	Replace the terminal loop with a browser-based chat interface	High — improves usability significantly
Conversation Memory	Pass previous Q&A pairs as context so follow-up questions work naturally	High — enables multi-turn conversations
PDF/Web Scraping Support	Auto-fetch documents from college website URLs into the knowledge base	Medium — reduces manual file management
Cross-encoder Reranking	Add a reranking step after retrieval to improve chunk selection accuracy	Medium — improves answer precision
HF Token Authentication	Set HF_TOKEN environment variable to remove the unauthenticated warning	Low — removes warning, increases rate limits
Hybrid Search	Combine vector search with keyword (BM25) search for better recall	Medium — catches exact term matches
Evaluation Framework	Use RAGAS or TruLens to automatically score faithfulness and relevance	High — enables systematic quality measurement

15. Reflection Questions

1. Why is RAG useful for GenAI applications?

RAG solves the hallucination problem in large language models. A general-purpose LLM has no knowledge of institution-specific policies and would either refuse to answer or generate plausible-sounding but incorrect information. RAG grounds the model's responses in actual source documents, making the answers verifiable and accurate. It also allows the knowledge base to be updated without retraining the model — simply re-running ingestion is sufficient.

2. What challenges did you face while preparing the data?

The main challenge was ensuring the documents had enough content for meaningful chunking. Initially, each file had only one paragraph, resulting in 6 total chunks and poor retrieval. Expanding each document to 5 structured sections with 300-400 words resolved this. Another challenge was consistent formatting — section headings made the natural language structure clearer for the text splitter.

3. How did your chunking strategy affect the quality of answers?

The chunk size had a direct and significant effect on answer quality. With 500-character chunks and 50-character overlap, each chunk captured enough context for the LLM to produce accurate and complete answers. The RecursiveCharacterTextSplitter preserved natural language boundaries within each chunk, and the 50-character overlap ensured that sentences were never truncated at chunk boundaries, making retrieved context coherent and usable.

4. What happened when the answer was not present in the knowledge base?

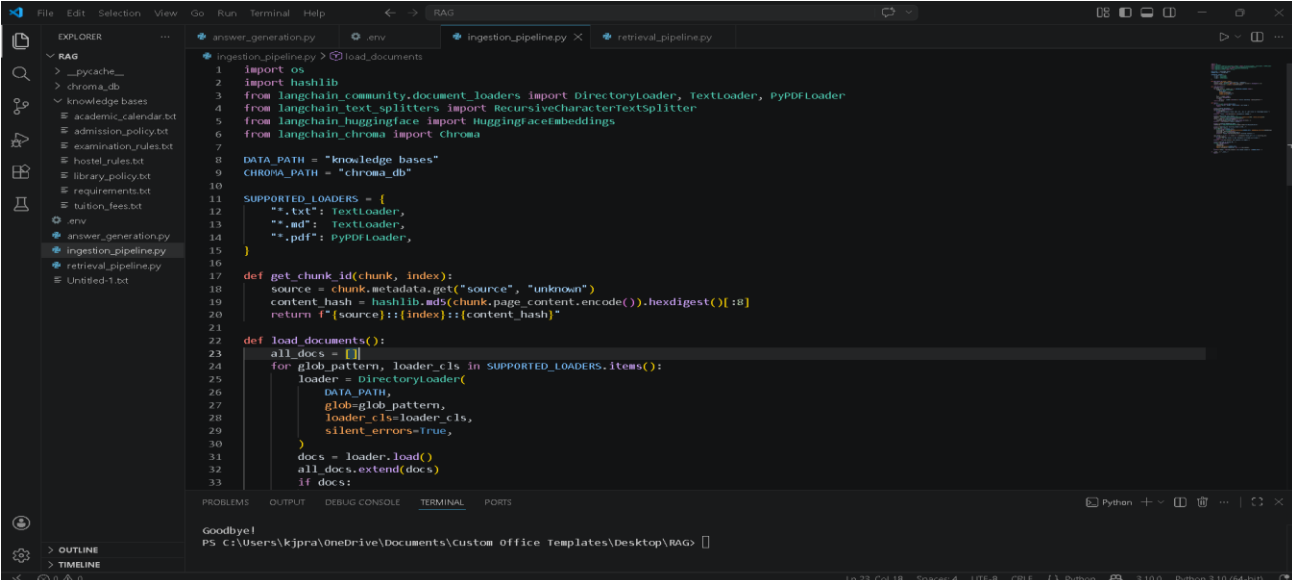
When a question had no relevant document (e.g., 'Is gym membership provided?'), two things happened: the retrieval pipeline either returned no chunks above the distance threshold, or returned low-relevance chunks from unrelated topics. In both cases, the prompt instruction ('If the answer isn't in the context, say Information not found in documents') caused the LLM to correctly respond that the information was unavailable, rather than hallucinating an answer. This is one of the core safety features of the RAG approach.

5. How can your application be improved in the future?

The most impactful immediate improvement would be a Streamlit web interface, which would make the application accessible to non-technical users such as students and staff without requiring terminal access. Adding conversation memory would enable multi-turn interactions where follow-up questions (e.g., 'What about for international students?') would be understood in context. For a production deployment, the knowledge base should be connected directly to the college's document management system so policy updates are automatically reflected without manual file management.

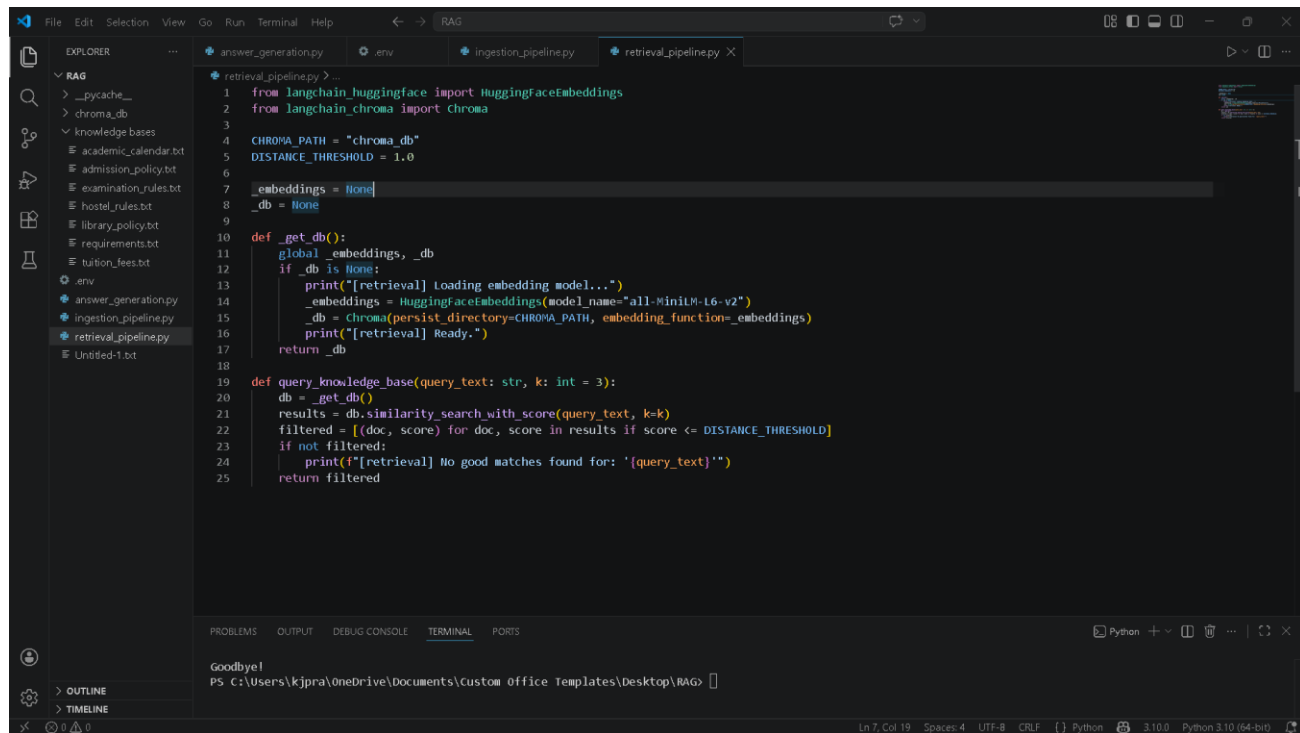
6.SCREEN SHOTS FROM VS CODE

INGESTION_PIPELINE



```
1 import os
2 import hashlib
3 from langchain_community.document_loaders import DirectoryLoader, TextLoader, PyPDFLoader
4 from langchain_text_splitters import RecursiveCharacterTextSplitter
5 from langchain_huggingface import HuggingFaceEmbeddings
6 from langchain_chroma import Chroma
7
8 DATA_PATH = "knowledge bases"
9 CHROMA_PATH = "chroma_db"
10
11
12 SUPPORTED_LOADERS = {
13     "*.txt": TextLoader,
14     "*.md": TextLoader,
15     "*.pdf": PyPDFLoader,
16 }
17
18 def get_chunk_id(chunk, index):
19     source = chunk.metadata.get("source", "unknown")
20     content_hash = hashlib.md5(chunk.page_content.encode()).hexdigest()[:8]
21     return f"{source}:{index}:{content_hash}"
22
23 def load_documents():
24     all_docs = []
25     for glob_pattern, loader_cls in SUPPORTED_LOADERS.items():
26         loader = DirectoryLoader(
27             DATA_PATH,
28             glob=glob_pattern,
29             loader_cls=loader_cls,
30             silent_errors=True,
31         )
32         docs = loader.load()
33         all_docs.extend(docs)
34         if docs:
35             print(f"Loaded {len(docs)} documents from {glob_pattern}")
36
37 if __name__ == "__main__":
38     load_documents()
39     print("Done!")
```

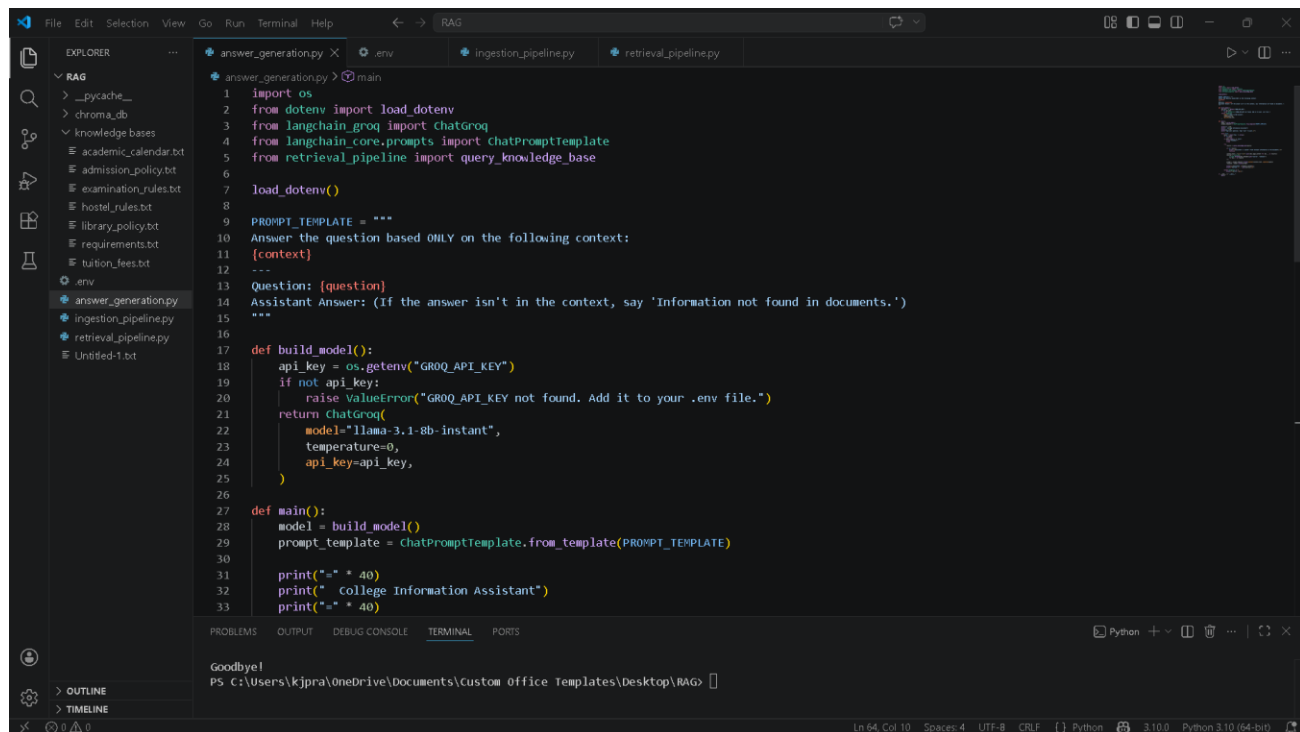
RETRIVAL_PIPELINE



```
1 from langchain_huggingface import HuggingFaceEmbeddings
2 from langchain_chroma import Chroma
3
4 CHROMA_PATH = "chroma_db"
5 DISTANCE_THRESHOLD = 1.0
6
7 _embeddings = None
8 _db = None
9
10 def _get_db():
11     global _embeddings, _db
12     if _db is None:
13         print("[retrieval] Loading embedding model...")
14         _embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
15         _db = Chroma(persist_directory=CHROMA_PATH, embedding_function=_embeddings)
16         print("[retrieval] Ready.")
17     return _db
18
19 def query_knowledge_base(query_text: str, k: int = 3):
20     db = _get_db()
21     results = db.similarity_search_with_score(query_text, k=k)
22     filtered = [(doc, score) for doc, score in results if score <= DISTANCE_THRESHOLD]
23     if not filtered:
24         print(f"[retrieval] No good matches found for: '{query_text}'")
25     return filtered
```

Goodbye!
PS C:\Users\kjp\OneDrive\Documents\Custom Office Templates\Desktop\RAG

ANSWER_GENERATION



```
1 import os
2 from dotenv import load_dotenv
3 from langchain_groq import ChatGroq
4 from langchain_core.prompts import ChatPromptTemplate
5 from retrieval_pipeline import query_knowledge_base
6
7 load_dotenv()
8
9 PROMPT_TEMPLATE = """
10 Answer the question based ONLY on the following context:
11 {context}
12 ---
13 Question: {question}
14 Assistant Answer: (If the answer isn't in the context, say 'Information not found in documents.')
15 """
16
17 def build_model():
18     api_key = os.getenv("GROQ_API_KEY")
19     if not api_key:
20         raise ValueError("GROQ_API_KEY not found. Add it to your .env file.")
21     return ChatGroq(
22         model="llama-3.1-8b-instant",
23         temperature=0,
24         api_key=api_key,
25     )
26
27 def main():
28     model = build_model()
29     prompt_template = ChatPromptTemplate.from_template(PROMPT_TEMPLATE)
30
31     print("* * * 40")
32     print("  College Information Assistant")
33     print("* * * 40")
```

Goodbye!
PS C:\Users\kjp\OneDrive\Documents\Custom Office Templates\Desktop\RAG

USED A GROQ API KEY HERE

KNOWLEDGE BASES :

[ACADEMIC CALENDAR , ADMISSION POLICY , EXAMINATION RULES AND REGULATIONS , HOSTEL AND RESIDENTIAL RULES , LIBRARY POLICY - CENTRAL LIBRARY , TUITION AND FEES]

ACADEMIC CALENDAR - 2026-2027

FALL SEMESTER 2026

The Fall Semester begins on September 1, 2026. Orientation week for new students runs from August 25 to August 31. The Add/Drop period for course registration ends on September 15. Mid-term examinations are scheduled for the third week of October. The last day of classes is December 12, 2026.

WINTER BREAK

Winter break is scheduled to start on December 20, 2026. The campus remains open for administrative services during this period. The campus reopens for the Spring Semester on January 5, 2027.

SPRING SEMESTER 2027

The Spring Semester begins on January 6, 2027. The Add/Drop period ends on January 20. Spring mid-term examinations are scheduled for the third week of March. Final examinations for the Spring Semester will be held in the first two weeks of May 2027.

SUMMER SESSION

A short optional Summer Session runs from June 1 to July 15, 2027. Students may enroll in a maximum of two courses during the Summer Session. Summer Session grades are included in the cumulative GPA calculation.

IMPORTANT DATES

- May 1, 2026: Admission cycle opens
- August 25, 2026: New student orientation begins
- September 1, 2026: Fall Semester begins
- December 20, 2026: Winter break starts
- January 5, 2027: Campus reopens
- January 6, 2027: Spring Semester begins

ADMISSION POLICY - 2026 ACADEMIC SESSION

The formal admission cycle for the 2026 academic session officially commences on May 1, 2026.

APPLICATION PROCESS

All applicants must submit a completed online application form through the official college portal. A non-refundable processing fee of \$50 is required at the time of submission. Incomplete applications will not be considered.

REQUIRED DOCUMENTS

The following documents are mandatory for all applicants:

- Official high school diploma or equivalent certificate
- Certified academic transcripts from all previously attended institutions
- A valid government-issued photo ID such as a passport or national ID card
- Two passport-sized photographs
- A completed personal statement of at least 300 words

INTERNATIONAL APPLICANTS

International students must provide a minimum TOEFL iBT score of 80 or an IELTS score of 6.5 to demonstrate English language proficiency. All foreign academic documents must be translated into English and notarized before submission.

SELECTION CRITERIA

Admission decisions are based on academic performance, personal statement quality, and availability of seats. Shortlisted candidates will be called for an in-person or virtual interview. Final admission offers will be sent via email by June 15, 2026.

DEFERRALS AND WITHDRAWALS

Admitted students may request a one-year deferral by submitting a written request before July 1, 2026. Withdrawal after July 1 will result in forfeiture of the processing fee. Students who withdraw after August 1 will also forfeit 50% of the first semester tuition.

EXAMINATION RULES AND REGULATIONS

ATTENDANCE REQUIREMENT

Students must maintain a minimum attendance of 75% in each course to be eligible to sit for final examinations. Students falling below this threshold will be barred from the examination and awarded a failing grade for that course.

EXAMINATION HALL CONDUCT

A valid Student ID card must be presented at the entrance of the examination hall. Students must be seated 10 minutes before the examination begins. Late entry beyond 15 minutes after the start of the exam is not permitted.

PERMITTED MATERIALS

Only materials explicitly listed on the examination instruction sheet are allowed inside the hall. Calculators are permitted only for mathematics and engineering papers. Mobile phones and smartwatches must be switched off and kept outside the hall.

ACADEMIC DISHONESTY

Academic dishonesty including plagiarism or use of unauthorized electronic devices during tests will result in an immediate F grade and possible suspension. A second offense will lead to permanent expulsion from the institution. All cases of dishonesty are recorded in the student's permanent academic record.

GRADING AND RESULTS

Examination results are published within three weeks of the final examination date. Students may apply for re-evaluation of their answer scripts within 10 days of result publication by paying a fee of \$20 per paper. Grades are non-negotiable after the re-evaluation window closes.

HOSTEL AND RESIDENTIAL RULES

ACCOMMODATION FEES

The annual fee for a standard double-occupancy room is \$1,200. Single-occupancy rooms are available at \$1,800 per year. All fees must be paid before the start of each academic year.

CURFEW AND SECURITY

Residential students are required to adhere to a nightly curfew of 9:00 PM to ensure campus safety. Students returning after curfew must report to the security desk and sign the late entry register. Repeated curfew violations will result in disciplinary action.

GUEST POLICY

Overnight guests are strictly prohibited unless a written request is approved by the Office of the Warden at least 48 hours in advance. Day visitors are allowed between 9:00 AM and 6:00 PM only. Visitors must sign in at the front desk and wear a visitor badge at all times.

ROOM RULES

Cooking inside dorm rooms using electric heaters or gas stoves is forbidden due to fire safety regulations. Students may use designated common kitchen areas between 7:00 AM and 10:00 PM. Loud music and noise above permissible levels is prohibited after 10:00 PM.

MAINTENANCE AND CLEANLINESS

Students are responsible for maintaining cleanliness in their rooms. Room inspections are conducted every two weeks by the hostel warden. Students found violating cleanliness standards will be issued a warning, and repeated violations may result in eviction from the hostel.

LIBRARY POLICY - CENTRAL LIBRARY

OPERATING HOURS

The Central Library is open daily from 8:00 AM to 9:00 PM during the regular semester. During examination periods the library extends its hours to 11:00 PM. The library is closed on public holidays and during winter and summer breaks.

BORROWING RULES

Students may borrow a maximum of 3 books simultaneously for a duration of 14 days. Books may be renewed once online through the library portal if no other student has reserved the same title. Reference books and journals are available for in-library use only and cannot be borrowed.

FINES AND PENALTIES

A late return fine of \$1 per day per book is charged for items not returned by the due date. Lost or damaged books must be replaced at the student's expense or a replacement cost of three times the book's market value will be charged. Repeated violations may result in suspension of borrowing privileges.

CONDUCT IN THE LIBRARY

Consumption of food and sweetened beverages is strictly prohibited in the reading zones. Mobile phones must be kept on silent mode at all times. Group discussions are permitted only in the designated group study rooms, which can be booked through the library front desk.

DIGITAL RESOURCES

Students have 24-hour access to the library's digital portal which includes e-books, research journals, and academic databases. Login credentials are provided at the time of enrollment. Sharing of login credentials with non-students is a violation of library policy and may result in account suspension.

TUITION AND FEES - 2026 ACADEMIC YEAR

TUITION RATES

Standard tuition for full-time undergraduate students is \$5,000 per semester. Part-time students taking fewer than 3 units are charged \$800 per unit. Graduate program tuition is \$6,500 per semester.

PAYMENT SCHEDULE

Tuition payments are due by the 10th of each calendar month. A grace period is provided until the 15th of the month. After the 15th, a flat late payment fine of \$100 is automatically applied to the student account.

ADDITIONAL FEES

Laboratory fees for science and engineering courses are an additional \$250 per unit. The student activity fee is \$75 per semester and covers access to sports facilities and student clubs. Library membership is included in the tuition fee at no extra cost.

SCHOLARSHIPS AND FINANCIAL AID

Merit-based scholarships are available for students with a GPA of 3.8 or above. Need-based financial aid applications must be submitted before April 30, 2026. Scholarship recipients must maintain a minimum GPA of 3.5 to retain their award each semester.

REFUND POLICY

Students who withdraw within the first two weeks of the semester are eligible for a 75% tuition refund. Withdrawals between weeks three and five qualify for a 50% refund. No refund is issued after the fifth week of the semester.

RAW CODE: INGESTION_PIPELINE:

```
import os

import hashlib

from langchain_community.document_loaders import DirectoryLoader,
TextLoader, PyPDFLoader

from langchain_text_splitters import RecursiveCharacterTextSplitter

from langchain_huggingface import HuggingFaceEmbeddings
```

```
from langchain_chroma import Chroma

DATA_PATH = "knowledge bases"
CHROMA_PATH = "chroma_db"

SUPPORTED_LOADERS = {
    "*.txt": TextLoader,
    "*.md": TextLoader,
    "*.pdf": PyPDFLoader,
}

def get_chunk_id(chunk, index):
    source = chunk.metadata.get("source", "unknown")
    content_hash =
hashlib.md5(chunk.page_content.encode()).hexdigest()[:8]
    return f"{source}::{index}::{content_hash}"

def load_documents():
    all_docs = []
    for glob_pattern, loader_cls in SUPPORTED_LOADERS.items():
        loader = DirectoryLoader(
            DATA_PATH,
            glob=glob_pattern,
            loader_cls=loader_cls,
            silent_errors=True,
        )
        docs = loader.load()
        all_docs.extend(docs)
        if docs:
            print(f" Loaded {len(docs)} file(s) matching
'{glob_pattern}'")
    return all_docs
```

```
def main():
    if not os.path.exists(DATA_PATH):
        print(f"Error: Folder '{DATA_PATH}' not found.")
        return

    print("Loading documents...")
    documents = load_documents()
    if not documents:
        print("No documents found. Add .txt, .md, or .pdf files to
'knowledge bases/'")
        return

    print(f"  Total: {len(documents)} document(s) loaded.")

    print("\nSplitting into chunks...")
    splitter = RecursiveCharacterTextSplitter(chunk_size=500,
chunk_overlap=50)
    chunks = splitter.split_documents(documents)
    for i, chunk in enumerate(chunks):
        chunk.metadata["chunk_id"] = get_chunk_id(chunk, i)
    print(f"  {len(chunks)} chunks created.")

    print("\nLoading embedding model...")
    embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")

    print("\nChecking for existing chunks in DB...")
    existing_ids = set()
    if os.path.exists(CHROMA_PATH):
        existing_db = Chroma(persist_directory=CHROMA_PATH,
embedding_function=embeddings)
        existing = existing_db.get()
        existing_ids = set(existing["ids"])
        print(f"  Found {len(existing_ids)} existing chunk(s).")
```

```
    new_chunks = [c for c in chunks if c.metadata["chunk_id"] not in
existing_ids]
    if not new_chunks:
        print("\nNo new chunks to add. Database is already up to date.")
        return
    print(f"    {len(new_chunks)} new chunk(s) to ingest.")

    print("\nSaving to Chroma DB...")
    Chroma.from_documents(
        new_chunks,
        embeddings,
        persist_directory=CHROMA_PATH,
        ids=[c.metadata["chunk_id"] for c in new_chunks],
    )
    print(f"\nDone. {len(new_chunks)} new chunks saved to
'{CHROMA_PATH}'.")

if __name__ == "__main__":
    main()
```

RETRIVAL_PIPELINE:

```
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_chroma import Chroma

CHROMA_PATH = "chroma_db"
DISTANCE_THRESHOLD = 1.0

_embeddings = None
_db = None

def _get_db():
    global _embeddings, _db
    if _db is None:
```

```
        print("[retrieval] Loading embedding model...")
        _embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-
v2")

        _db = Chroma(persist_directory=CHROMA_PATH,
embedding_function=_embeddings)

        print("[retrieval] Ready.")

    return _db

def query_knowledge_base(query_text: str, k: int = 3):
    db = _get_db()
    results = db.similarity_search_with_score(query_text, k=k)
    filtered = [(doc, score) for doc, score in results if score <=
DISTANCE_THRESHOLD]
    if not filtered:
        print(f"[retrieval] No good matches found for: '{query_text}'")
    return filtered
```

ANSWER_GENERATION:

```
import os
from dotenv import load_dotenv
from langchain_groq import ChatGroq
from langchain_core.prompts import ChatPromptTemplate
from retrieval_pipeline import query_knowledge_base

load_dotenv()

PROMPT_TEMPLATE = """
Answer the question based ONLY on the following context:
{context}
---
Question: {question}
Assistant Answer: (If the answer isn't in the context, say 'Information
not found in documents.')
"""
```

```
def build_model():
    api_key = os.getenv("GROQ_API_KEY")
    if not api_key:
        raise ValueError("GROQ_API_KEY not found. Add it to your .env
file.")
    return ChatGroq(
        model="llama-3.1-8b-instant",
        temperature=0,
        api_key=api_key,
    )

def main():
    model = build_model()
    prompt_template = ChatPromptTemplate.from_template(PROMPT_TEMPLATE)

    print("=" * 40)
    print("  College Information Assistant")
    print("=" * 40)
    print("Type your question. Type 'exit' to quit.\n")

    while True:
        query = input("You: ").strip()
        if not query:
            continue
        if query.lower() == "exit":
            print("Goodbye!")
            break

        try:
            results = query_knowledge_base(query)

            if not results:
                print("\nAssistant: I couldn't find relevant information
in the documents.\n")
                continue
```

```
        context_text = "\n\n---\n\n".join([doc.page_content for doc,
_ in results])
        sources = list(set([
            os.path.basename(doc.metadata.get("source", "unknown"))
            for doc, _ in results
        ]))

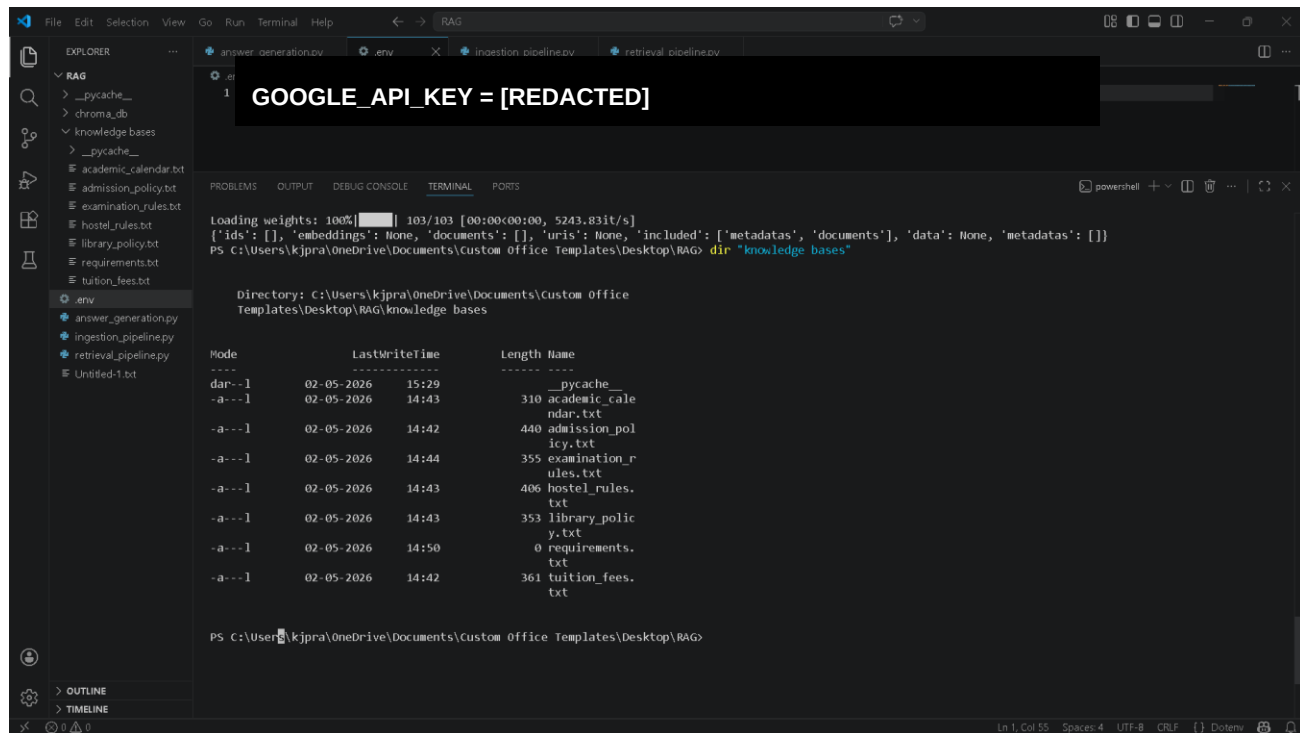
        prompt = prompt_template.format(context=context_text,
question=query)
        response = model.invoke(prompt)

        print(f"\nAssistant: {response.content}")
        print(f"Sources: {' '.join(sources)}\n")

    except Exception as e:
        print(f"\nError: {e}\n")

if __name__ == "__main__":
    main()
```

VS_CODE SCREENSHOTS:



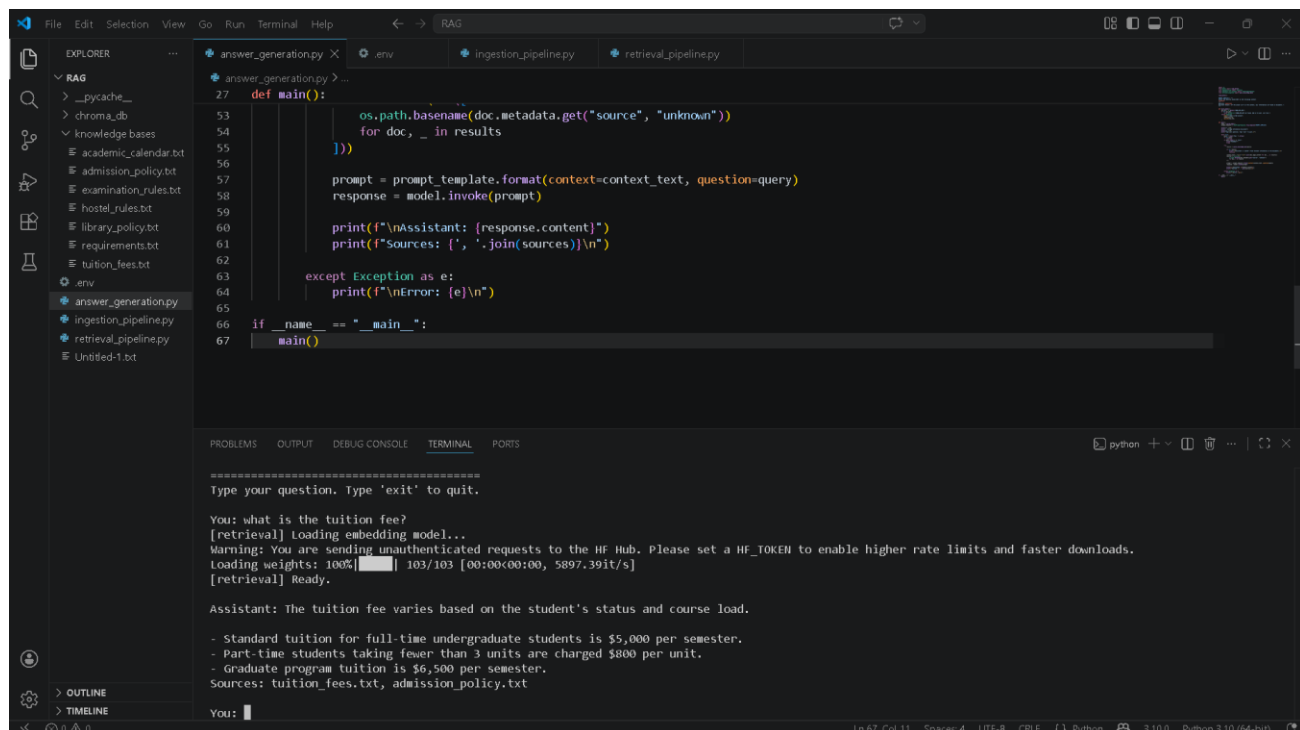
```
1 GOOGLE_API_KEY = [REDACTED]
```

```
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 5243.83it/s]
{'ids': [], 'embeddings': None, 'documents': [], 'uris': None, 'included': ['metadata', 'documents'], 'data': None, 'metadata': []}
PS C:\Users\kjp\OneDrive\Documents\Custom Office Templates\Desktop\RAG> dir "knowledge_bases"

Directory: C:\Users\kjp\OneDrive\Documents\Custom Office Templates\Desktop\RAG\knowledge_bases

Mode                LastWriteTime         Length Name
----                -
dar--1             02-05-2026   15:29             __pycache__
-a---1             02-05-2026   14:43             310 academic_cale
ndar.txt
-a---1             02-05-2026   14:42             440 admission_pol
icy.txt
-a---1             02-05-2026   14:44             355 examination_r
ules.txt
-a---1             02-05-2026   14:43             406 hostel_rules.
txt
-a---1             02-05-2026   14:43             353 library_polic
y.txt
-a---1             02-05-2026   14:50              0 requirements.
txt
-a---1             02-05-2026   14:42             361 tuition_fees.
txt

PS C:\Users\kjp\OneDrive\Documents\Custom Office Templates\Desktop\RAG>
```



```
def main():
    os.path.basename(doc.metadata.get("source", "unknown"))
    for doc, _ in results
    ])

    prompt = prompt_template.format(context=context_text, question=query)
    response = model.invoke(prompt)

    print(f"\nAssistant: {response.content}")
    print(f"Sources: {' '.join(sources)}\n")

except Exception as e:
    print(f"\nError: {e}\n")

if __name__ == "__main__":
    main()
```

```
=====
Type your question. Type 'exit' to quit.

You: what is the tuition fee?
[retrieval] Loading embedding model...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 5897.39it/s]
[retrieval] Ready.

Assistant: The tuition fee varies based on the student's status and course load.

- Standard tuition for full-time undergraduate students is $5,000 per semester.
- Part-time students taking fewer than 3 units are charged $800 per unit.
- Graduate program tuition is $6,500 per semester.
Sources: tuition_fees.txt, admission_policy.txt

You: 
```



```
File Edit Selection View Go Run Terminal Help
RAG
answer_generation.py X .env ingestion_pipeline.py retrieval_pipeline.py
27 def main():
53     os.path.basename(doc.metadata.get("source", "unknown"))
54     for doc, _ in results
55     )))
56
57     prompt = prompt_template.format(context=context_text, question=query)
58     response = model.invoke(prompt)
59
60     print(f"\nAssistant: {response.content}")
61     print(f"Sources: {' '.join(sources)}\n")
62
63     except Exception as e:
64         print(f"\nError: {e}\n")
65
66 if __name__ == "__main__":
67     main()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
python + - - - - -
- Standard tuition for full-time undergraduate students is $5,000 per semester.
- Part-time students taking fewer than 3 units are charged $800 per unit.
- Graduate program tuition is $6,500 per semester.
Sources: tuition_fees.txt, admission_policy.txt

You: what is the application fee?

Assistant: The application fee is $50.
Sources: admission_policy.txt

You: what happens if i cheat in an exam?

Assistant: If you cheat in an exam, you will receive an immediate F grade. Additionally, you may face possible suspension.
Sources: examination_rules.txt

You: 
```

```
File Edit Selection View Go Run Terminal Help
RAG
answer_generation.py X .env ingestion_pipeline.py retrieval_pipeline.py
27 def main():
53     os.path.basename(doc.metadata.get("source", "unknown"))
54     for doc, _ in results
55     )))
56
57     prompt = prompt_template.format(context=context_text, question=query)
58     response = model.invoke(prompt)
59
60     print(f"\nAssistant: {response.content}")
61     print(f"Sources: {' '.join(sources)}\n")
62
63     except Exception as e:
64         print(f"\nError: {e}\n")
65
66 if __name__ == "__main__":
67     main()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
python + - - - - -
You: what is the application fee?

Assistant: The application fee is $50.
Sources: admission_policy.txt

You: what happens if i cheat in an exam?

Assistant: If you cheat in an exam, you will receive an immediate F grade. Additionally, you may face possible suspension.
Sources: examination_rules.txt

You: can i cook in my dorm room?

Assistant: No, you cannot cook in your dorm room. According to the room rules, cooking inside dorm rooms using electric heaters or gas stoves is forbidden due to fire safety regulations.
Sources: hostel_rules.txt

You: 
```

```
File Edit Selection View Go Run Terminal Help
RAG
answer_generation.py X .env ingestion_pipeline.py retrieval_pipeline.py
def main():
    27
    53     os.path.basename(doc.metadata.get("source", "unknown"))
    54     for doc, _ in results
    55     )))
    56
    57     prompt = prompt_template.format(context=context_text, question=query)
    58     response = model.invoke(prompt)
    59
    60     print(f"\nAssistant: {response.content}")
    61     print(f"Sources: {' '.join(sources)}\n")
    62
    63     except Exception as e:
    64         print(f"\nError: {e}\n")
    65
    66 if __name__ == "__main__":
    67     main()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
python + - - - - -

You: what happens if i cheat in an exam?
Assistant: If you cheat in an exam, you will receive an immediate F grade. Additionally, you may face possible suspension.
Sources: examination_rules.txt

You: can i cook in my dorm room?
Assistant: No, you cannot cook in your dorm room. According to the room rules, cooking inside dorm rooms using electric heaters or gas stoves is forbidden due to fire safety regulations.
Sources: hostel_rules.txt

You: what is the fine for returning books late?
Assistant: The fine for returning books late is $1 per day per book.
Sources: library_policy.txt

You: 
```

```
File Edit Selection View Go Run Terminal Help
RAG
answer_generation.py X .env ingestion_pipeline.py retrieval_pipeline.py
def main():
    27
    53     os.path.basename(doc.metadata.get("source", "unknown"))
    54     for doc, _ in results
    55     )))
    56
    57     prompt = prompt_template.format(context=context_text, question=query)
    58     response = model.invoke(prompt)
    59
    60     print(f"\nAssistant: {response.content}")
    61     print(f"Sources: {' '.join(sources)}\n")
    62
    63     except Exception as e:
    64         print(f"\nError: {e}\n")
    65
    66 if __name__ == "__main__":
    67     main()

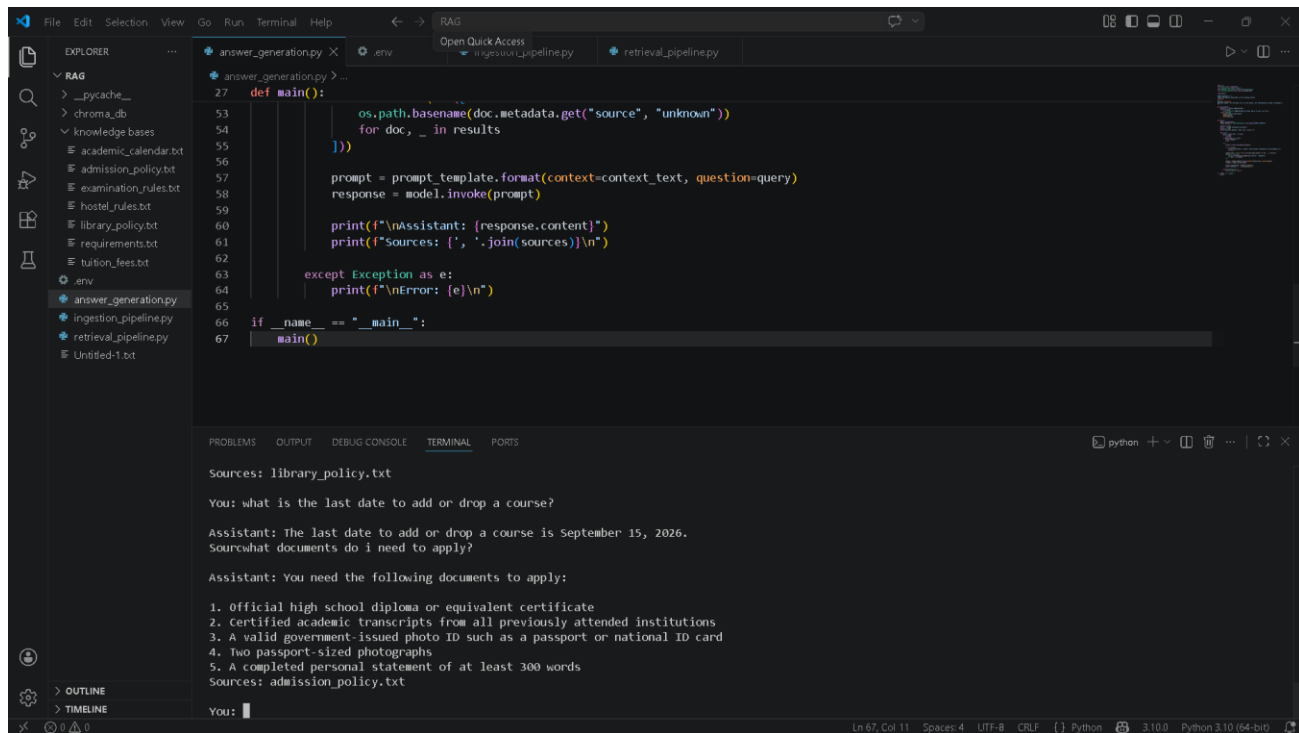
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
python + - - - - -

You: can i cook in my dorm room?
Assistant: No, you cannot cook in your dorm room. According to the room rules, cooking inside dorm rooms using electric heaters or gas stoves is forbidden due to fire safety regulations.
Sources: hostel_rules.txt

You: what is the fine for returning books late?
Assistant: The fine for returning books late is $1 per day per book.
Sources: library_policy.txt

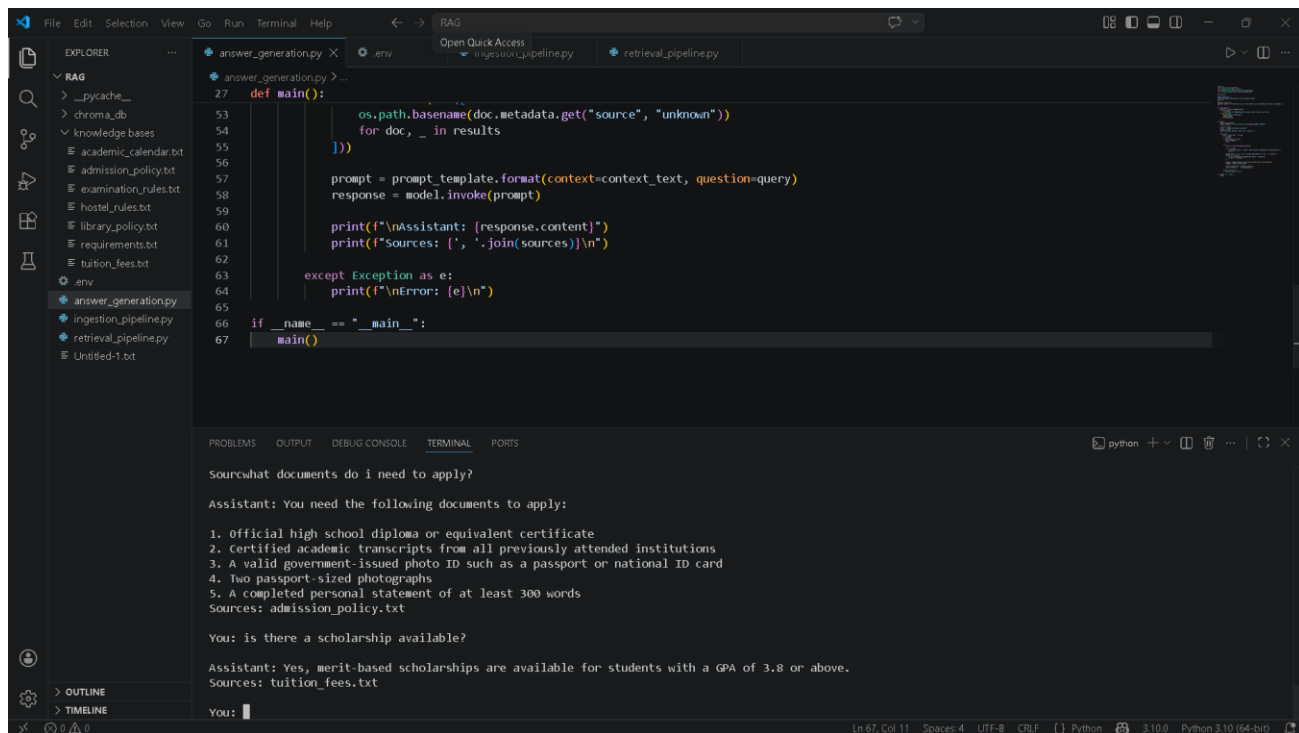
You: what is the last date to add or drop a course?
Assistant: The last date to add or drop a course is September 15, 2026.
Sources: academic_calendar.txt

You: 
```



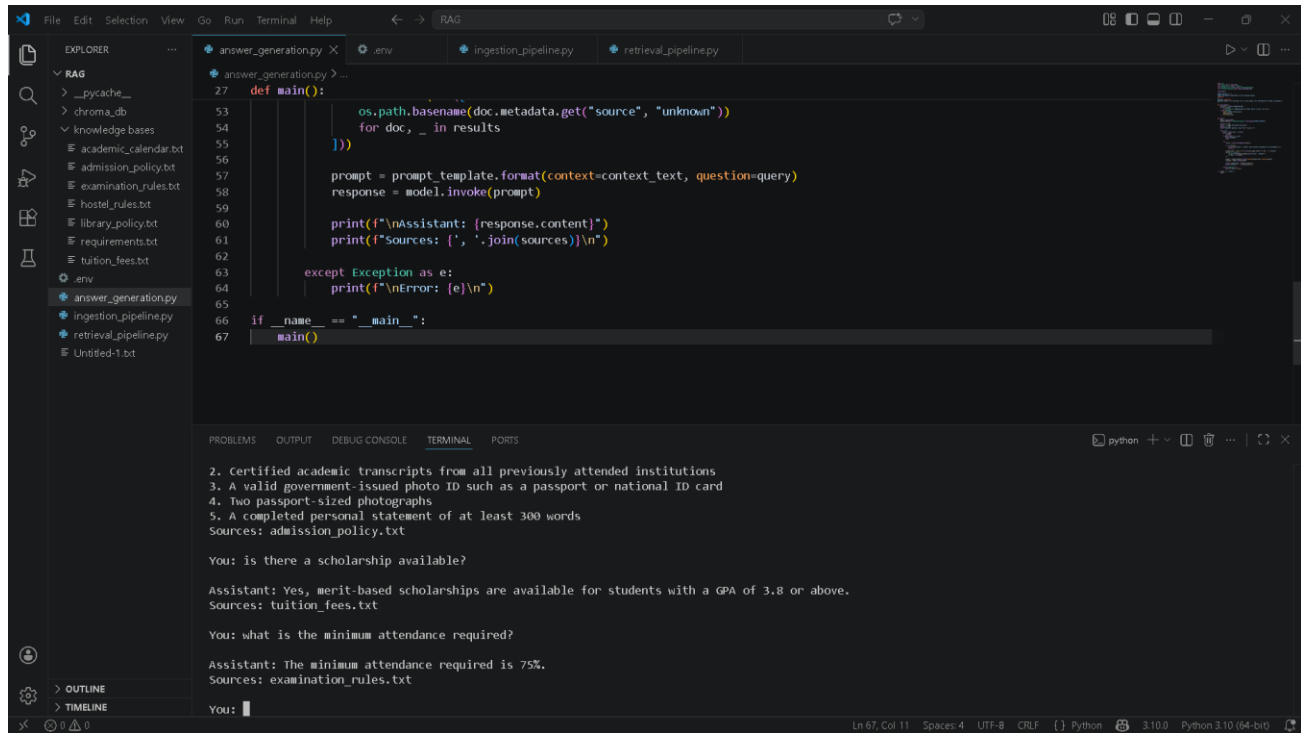
```
File Edit Selection View Go Run Terminal Help
RAG
answer_generation.py X .env ingestion_pipeline.py retrieval_pipeline.py
27 def main():
53     os.path.basename(doc.metadata.get("source", "unknown"))
54     for doc, _ in results
55     ])
56
57     prompt = prompt_template.format(context=context_text, question=query)
58     response = model.invoke(prompt)
59
60     print(f"\nAssistant: {response.content}")
61     print(f"Sources: {' '.join(sources)}\n")
62
63     except Exception as e:
64         print(f"\nError: {e}\n")
65
66 if __name__ == "__main__":
67     main()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Sources: library_policy.txt
You: what is the last date to add or drop a course?
Assistant: The last date to add or drop a course is September 15, 2026.
Source: what documents do i need to apply?
Assistant: You need the following documents to apply:
1. Official high school diploma or equivalent certificate
2. Certified academic transcripts from all previously attended institutions
3. A valid government-issued photo ID such as a passport or national ID card
4. Two passport-sized photographs
5. A completed personal statement of at least 300 words
Sources: admission_policy.txt
You:
```



```
File Edit Selection View Go Run Terminal Help
RAG
answer_generation.py X .env ingestion_pipeline.py retrieval_pipeline.py
27 def main():
53     os.path.basename(doc.metadata.get("source", "unknown"))
54     for doc, _ in results
55     ])
56
57     prompt = prompt_template.format(context=context_text, question=query)
58     response = model.invoke(prompt)
59
60     print(f"\nAssistant: {response.content}")
61     print(f"Sources: {' '.join(sources)}\n")
62
63     except Exception as e:
64         print(f"\nError: {e}\n")
65
66 if __name__ == "__main__":
67     main()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Source: what documents do i need to apply?
Assistant: You need the following documents to apply:
1. Official high school diploma or equivalent certificate
2. Certified academic transcripts from all previously attended institutions
3. A valid government-issued photo ID such as a passport or national ID card
4. Two passport-sized photographs
5. A completed personal statement of at least 300 words
Sources: admission_policy.txt
You: is there a scholarship available?
Assistant: Yes, merit-based scholarships are available for students with a GPA of 3.8 or above.
Sources: tuition_fees.txt
You:
```



```
File Edit Selection View Go Run Terminal Help
RAG
answer_generation.py X .env ingestion_pipeline.py retrieval_pipeline.py
answer_generation.py > ...
27 def main():
53     os.path.basename(doc.metadata.get("source", "unknown"))
54     for doc, _ in results
55     )))
56
57     prompt = prompt_template.format(context=context_text, question=query)
58     response = model.invoke(prompt)
59
60     print(f"\nAssistant: {response.content}")
61     print(f"Sources: {' '.join(sources)}\n")
62
63     except Exception as e:
64         print(f"\nError: {e}\n")
65
66 if __name__ == "__main__":
67     main()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
python + - - - - -
2. Certified academic transcripts from all previously attended institutions
3. A valid government-issued photo ID such as a passport or national ID card
4. Two passport-sized photographs
5. A completed personal statement of at least 300 words
Sources: admission_policy.txt

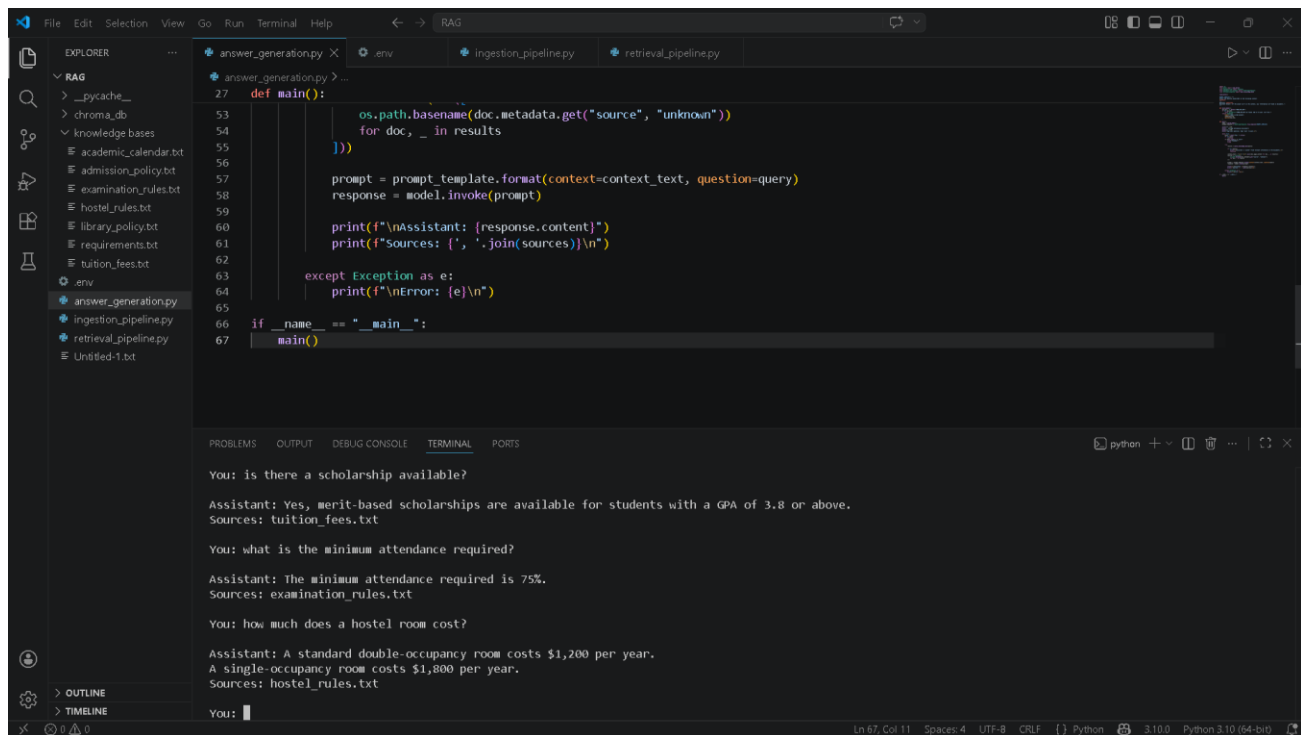
You: is there a scholarship available?

Assistant: Yes, merit-based scholarships are available for students with a GPA of 3.8 or above.
Sources: tuition_fees.txt

You: what is the minimum attendance required?

Assistant: The minimum attendance required is 75%.
Sources: examination_rules.txt

You: 
```



```
File Edit Selection View Go Run Terminal Help
RAG
answer_generation.py X .env ingestion_pipeline.py retrieval_pipeline.py
answer_generation.py > ...
27 def main():
53     os.path.basename(doc.metadata.get("source", "unknown"))
54     for doc, _ in results
55     )))
56
57     prompt = prompt_template.format(context=context_text, question=query)
58     response = model.invoke(prompt)
59
60     print(f"\nAssistant: {response.content}")
61     print(f"Sources: {' '.join(sources)}\n")
62
63     except Exception as e:
64         print(f"\nError: {e}\n")
65
66 if __name__ == "__main__":
67     main()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
python + - - - - -
You: is there a scholarship available?

Assistant: Yes, merit-based scholarships are available for students with a GPA of 3.8 or above.
Sources: tuition_fees.txt

You: what is the minimum attendance required?

Assistant: The minimum attendance required is 75%.
Sources: examination_rules.txt

You: how much does a hostel room cost?

Assistant: A standard double-occupancy room costs $1,200 per year.
A single-occupancy room costs $1,800 per year.
Sources: hostel_rules.txt

You: 
```

```
File Edit Selection View Go Run Terminal Help
RAG
EXPLORER
  RAG
    > _pyscache_
    > chroma_db
    > knowledge bases
    academic_calendar.txt
    admission_policy.txt
    examination_rules.txt
    hostel_rules.txt
    library_policy.txt
    requirements.txt
    tuition_fees.txt
    .env
    answer_generation.py
    ingestion_pipeline.py
    retrieval_pipeline.py
    Untitled-1.txt
  OUTLINE
  TIMELINE

answer_generation.py
27 def main():
53     os.path.basename(doc.metadata.get("source", "unknown"))
54     for doc, _ in results
55     ])
56
57     prompt = prompt_template.format(context=context_text, question=query)
58     response = model.invoke(prompt)
59
60     print(f"\nAssistant: {response.content}")
61     print(f"Sources: {' '.join(sources)}\n")
62
63     except Exception as e:
64         print(f"\nError: {e}\n")
65
66 if __name__ == "__main__":
67     main()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
python
You: what is the minimum attendance required?
Assistant: The minimum attendance required is 75%.
Sources: examination_rules.txt

You: how much does a hostel room cost?
Assistant: A standard double-occupancy room costs $1,200 per year.
A single-occupancy room costs $1,800 per year.
Sources: hostel_rules.txt

You: what are the library timings?
[retrieval] No good matches found for: 'what are the library timings?'
Assistant: I couldn't find relevant information in the documents.

You:
```

```
File Edit Selection View Go Run Terminal Help
RAG
EXPLORER
  RAG
    > _pyscache_
    > chroma_db
    > knowledge bases
    academic_calendar.txt
    admission_policy.txt
    examination_rules.txt
    hostel_rules.txt
    library_policy.txt
    requirements.txt
    tuition_fees.txt
    .env
    answer_generation.py
    ingestion_pipeline.py
    retrieval_pipeline.py
    Untitled-1.txt
  OUTLINE
  TIMELINE

answer_generation.py
27 def main():
53     os.path.basename(doc.metadata.get("source", "unknown"))
54     for doc, _ in results
55     ])
56
57     prompt = prompt_template.format(context=context_text, question=query)
58     response = model.invoke(prompt)
59
60     print(f"\nAssistant: {response.content}")
61     print(f"Sources: {' '.join(sources)}\n")
62
63     except Exception as e:
64         print(f"\nError: {e}\n")
65
66 if __name__ == "__main__":
67     main()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
python
You: how much does a hostel room cost?
Assistant: A standard double-occupancy room costs $1,200 per year.
A single-occupancy room costs $1,800 per year.
Sources: hostel_rules.txt

You: what are the library timings?
[retrieval] No good matches found for: 'what are the library timings?'
Assistant: I couldn't find relevant information in the documents.

You: how many books can i borrow at once?
Assistant: You can borrow a maximum of 3 books simultaneously.
Sources: library_policy.txt

You:

Ln 67, Col 11 | Spaces: 4 | UTF-8 | CRLF | Python | 3.10.8 | Python 2.10 (64-bit)
```

```
File Edit Selection View Go Run Terminal Help
RAG
EXPLORER
RAG
  __pycache__
  chroma_db
  knowledge bases
    academic_calendar.txt
    admission_policy.txt
    examination_rules.txt
    hostel_rules.txt
    library_policy.txt
    requirements.txt
    tuition_fees.txt
  .env
  answer_generation.py
  ingestion_pipeline.py
  retrieval_pipeline.py
  Untitled-1.txt
OUTLINE
TIMELINE

You: what is the tuition fee?
[retrieval] loading embedding model...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%| 103/103 [00:00:00:00, 5897.39it/s]
[retrieval] Ready.

Assistant: The tuition fee varies based on the student's status and course load.

- Standard tuition for full-time undergraduate students is $5,000 per semester.
- Part-time students taking fewer than 3 units are charged $800 per unit.
- Graduate program tuition is $6,500 per semester.
Sources: tuition_fees.txt, admission_policy.txt

You: what is the application fee?
Assistant: The application fee is $50.
Sources: admission_policy.txt

You: what happens if i cheat in an exam?
Assistant: If you cheat in an exam, you will receive an immediate F grade. Additionally, you may face possible suspension.
Sources: examination_rules.txt

You: can i cook in my dorm room?
Assistant: No, you cannot cook in your dorm room. According to the room rules, cooking inside dorm rooms using electric heaters or gas stoves is forbidden due to fire safety regulations.
Sources: hostel_rules.txt

You: what is the fine for returning books late?
Assistant: The fine for returning books late is $1 per day per book.
Sources: library_policy.txt

You: what is the last date to add or drop a course?
Assistant: The last date to add or drop a course is September 15, 2026.
Source: what documents do i need to apply?

Assistant: You need the following documents to apply:

1. Official high school diploma or equivalent certificate
```

```
File Edit Selection View Go Run Terminal Help
RAG
EXPLORER
RAG
  __pycache__
  chroma_db
  knowledge bases
    academic_calendar.txt
    admission_policy.txt
    examination_rules.txt
    hostel_rules.txt
    library_policy.txt
    requirements.txt
    tuition_fees.txt
  .env
  answer_generation.py
  ingestion_pipeline.py
  retrieval_pipeline.py
  Untitled-1.txt
OUTLINE
TIMELINE

PS C:\Users\kjp\OneDrive\Documents\Custom Office Templates\Desktop\RAG> & c:\Users\kjp\AppData\Local\Programs\Python\Python310\python.exe "c:\Users\kjp\OneDrive\Documents\Custom Office Templates\Desktop\RAG\answer_generation.py"
College Information Assistant
Type your question. Type 'exit' to quit.

You: what is the curfew time in the hostel?
[retrieval] loading embedding model...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%| 103/103 [00:00:00:00, 6060.54it/s]
[retrieval] Ready.

Assistant: The curfew time in the hostel is 9:00 PM.
Sources: hostel_rules.txt

You: exit
Goodbye!
```

End of Report

Praveen K J | Business Analytics Certification | ASAP Kerala | 2026